

Hybrid Authoring Tool for Human-Robot Conversations

Evelyne Prodescu¹

Supervisor: Mike Ligthart ; Vrije University Amsterdam

Abstract. This thesis presents a hybrid multi-session narrative dialog system for human-robot interaction that combines a rule-based dialog manager with constrained large language model generation and persistent memory. The goal is to support coherent, personalized, multi-session conversations between social robots and users. The system was evaluated in two stages. An internal technical evaluation showed a 100% functional pass rate across 20 sessions, with all unrecognized inputs recovered through fallback handling and no unrecoverable errors. A subsequent expert evaluation involved interviewing six experts in hour-long sessions which were examined through thematic analysis. The analysis reveals that conversational breakdowns occur mainly during transitions, turn-taking, and intent interpretation, and that users interpret system failures through the robot’s narrative identity. These findings show that a structured hybrid system supports reliable multi-session interaction, but that conversational quality depends less on selecting relevant content than on how well those breakdown points are managed. The thesis contributes a working hybrid dialog system, a two-stage evaluation framework, and eight prioritized design recommendations for future improvements to the system.

Keywords: Human-robot interaction, dialog management, large language models, multi-session narrative dialog system, persistent memory

1 Introduction

In the hospital, a robot companion sits beside a child’s bed. In a classroom, a robot tutor waits for a student who returns to it, week after week. In settings like these, a social robot’s value comes from the relationship it builds across multiple sessions. Personalization and rapport emerge longitudinally, over weeks or months, not within a single session [13].

This is difficult because sustaining a relationship requires the robot and user to build shared common ground through their conversational history [11]. So, it remains a major challenge, as robots are not yet capable of spontaneous and contextually appropriate dialog.

A dialog system used in HRI should sustain user engagement across repeated sessions, and maintain coherent, narrative continuity. It should balance autonomy and designer control. Two competing approaches dominate social robot dialog design and address these properties only in isolation. Scripted systems

are robust and predictable, which is why they prevail in sensitive domains such as healthcare [12], but they cannot adapt beyond what was explicitly authored. Children’s social presence ratings rise at first but decline across repeated sessions as scripted structure becomes apparent [15]. Systems built around LLMs introduce flexibility but at the cost of reliability: they hallucinate [19], making them unsafe to use on their own [9]. Neither approach is sufficient alone.

To address this gap, this thesis presents a dialog management system that enables multi-session interaction through a combination of mini-dialogs (small, self-contained units of conversation) and constrained LLM generation. The thesis proceeds in three steps. First, we implement the system (section 4). Second, we evaluate it through an internal technical evaluation and expert interviews (sections 5,6). Third, we reflect on these findings to derive design implications for future improvements (section 7).

Together, these steps address the question that motivates the thesis. How can a multi-session narrative dialog system for social robots be designed, implemented, and evaluated so that it reliably supports coherent and personalized interaction, while meeting the needs of designers and developers?

2 Related work

2.1 Long-Term Engagement

Research on multi-session HRI identifies two central challenges: engagement fades once novelty wears off [10], and the benefits of adapting to a user’s interaction history are only visible longitudinally, emerging after months of interaction [13]. Tying the interaction to an ongoing past-and-future narrative can offset the conversational limitations of a scripted system. The Valerie robot receptionist [5] used a serial soap-opera storyline updated every two weeks and drew 254 visitors per week over nine months. Bickmore and Cassell [2] further demonstrate that small talk and conversational storytelling are necessary to establish continuity.

2.2 Memory-based personalisation

Persistent memory in HRI extends beyond recalling past interactions. It is a social mechanism through which relationships develop [17]. Children in Leite et al.’s study (2017) rate robots higher on intelligence and likeability when previously shared information is actively applied in conversation than those in a no-memory baseline study [16]. But personalisation has risks. Disclosing retained information that a user did not expect the robot to hold produces negative reactions. So, memory must be surfaced selectively and with judgment [14].

Wang et al. (2025), in the AnnaAgent multi-session memory system, demonstrate that dynamically updating user profiles across sessions (tracking changing preferences, emotional states, and conversation history) enables more contextually consistent long-term interaction than static user models [23].

2.3 LLM integration

Rule-based dialog systems outperform keyword-based or random alternatives on perceived coherence [6]. LLMs address the flexibility limitations of structured approaches, but introduce new ones. LLM output quality degrades signif-

icantly across multi-turn role-play dialog [20], while human-authored responses improve over the same turns [19]. In real-world deployment with older adults, LLM-powered companion robots produce frequent hallucinations and incoherent responses, indicating that LLMs alone are unsafe for HRI [9]. Safe integration requires two system boundaries: a validation layer to intercept hallucinations [21] and authored structure to manage cross-turn coherence, which is where current LLMs most consistently underperform [8].

2.4 Dialog authoring tools and workflows

Interactive narrative authoring research identifies transparency of story structure, iterative testing support, and management of branching complexity as the core UX requirements for authoring tools [7]. In collaborative storytelling between children and the iCub robot, unanchored LLM generation produced thematic incoherence, showing that narrative coherence requires an authoring structure that generative systems alone cannot provide [18]. Authoring for HRI also introduces constraints that existing narrative tools do not address. Robot storytellers are physically embodied. Their expressive and social dimensions are directly reflected in the listeners’ responses. The authoring framework must therefore support emotional arc, timing, and character cues that text-based tools treat as irrelevant [4].

Across these areas, a consistent gap emerges. Hybrid architectures are considered structurally unavoidable, yet existing systems rarely combine rule-based and generative dialog management in a single architecture with a corresponding authoring workflow [22]. This thesis aims to close this gap by implementing a hybrid system.

3 Aim and Objectives

The goal of this thesis is to evaluate the performance of a multi-session narrative dialog system and to understand how generative AI capabilities can be incorporated into a system originally designed in a pre-LLM era, while remaining faithful to its core design principle of facilitating designed interactions.

To achieve this aim, we set three objectives. First, implement the system. Second, evaluate the system: verify the technical reliability and integration of the conversation pipeline (dialog manager, authoring tool, text-to-speech engine, and robot interface) and assess the usability and extensibility of the system for interaction designers and developers, evaluating whether the system supports their needs. Third, reflect on the findings to identify how the system can be improved and what design implications can be derived for future narrative dialog systems that integrate structured dialog and LLM-based components.

The target population is experts who design and develop robot interactions, as they might benefit from using such a system in their research. To guide this study, the following research questions are addressed:

RQ1: How can a multi-session narrative dialog system for social robots be designed and implemented to support coherent, adaptive, and personalized interactions?

RQ2: How do expert users experience the usability and interaction workflow of the dialog management system?

RQ3: How can the system be improved and what design implications can be derived for future narrative dialog systems that integrate structured dialog and LLM-based components?

4 System design

In this section, we present the design choices and specifications of the multi-session narrative dialog system, thus addressing the first research question. The system is designed to fulfill the following requirements:

- 1) support narrative continuity across sessions, ensuring that previously completed story elements are not repeated unnecessarily;
- 2) select future mini-dialogs such that coherence is maintained within each session, ensuring a meaningful conversational progression;
- 3) balance robot guidance with user influence, allowing the robot to guide the interaction while adapting to user input and preferences;
- 4) support personalization through LLM integration and memory;
- 5) and remain extensible to new features.

These requirements motivate the hybrid design of the system. A key premise of the narrative dialog approach is that interaction quality and coherence depend on carefully authored content. This makes the authoring environment an essential part of the system, distinct from the dialog manager itself. We therefore introduce a hybrid authoring tool that combines structured, scripted dialog authoring with LLM-assisted content generation, allowing authors to maintain control over narrative structure.

4.1 Architecture overview

The system consists of five components that collectively support the interaction (see Figure 1). First, there is a collection of authored dialog content in the form of mini-dialogs. These mini-dialogs are selected and executed by the dialog runtime (green box in figure 1). Any information collected during the interaction is stored in the persistent memory component (purple box in figure 1). While executing the mini-dialogs, the dialog runtime can dynamically insert this information back into the conversation. The dialog content can be executed on a wide range of platforms, such as Robots like Nao, Pepper, Alphamini, Reachy Mini, or computing devices such as laptops. These devices are managed by a separate component called Platforms (red box in figure 1). Finally, there are external services: speech recognition, text-to-speech, natural language understanding and generation including LLMs (orange box in figure 1). These services can be called from the Dialog Manager.

Memory stores completed dialogs, user variables, topics of interest, and all session histories per participant, allowing the system to adapt while maintaining structure. User responses update memory during interaction. For example, the user’s preferences or interests are stored and used to influence the selection of future dialog. Topics of interest are aggregated over time to prioritize relevant

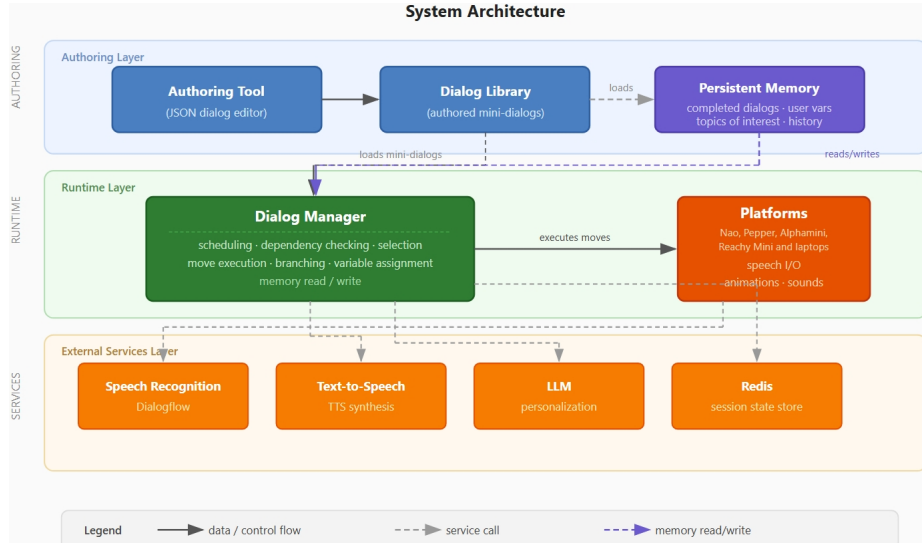


Fig. 1. Overview of the multi-session narrative dialog system architecture.

content. LLM-based components are used in a limited, controlled manner to personalize responses.

4.2 Dialog structure and moves

The dialog content is authored as mini-dialogs: the building blocks for the interaction, self-contained units of conversation. It can be seen as a paragraph in written text. We distinguish among three types of mini-dialogs. Functional dialogs provide structural elements such as greetings and farewells. Narrative dialogs represent story elements that unfold across multiple sessions. Chitchat dialogs provide short, topic-based interactions that introduce flexibility and support personalization.

Dialog execution is based on dialog moves, which are the smallest unit of interaction in the system. There are four types of moves. A **Say** move says the text to the participant. An **Ask** move requests user input, allowing for different response types: yes/no answers, options (presents an array of options), and open response (the user can reply with full sentences). Ask moves may include branching logic that determines how the dialog continues based on user responses. For example, a yes/no question leads to different subsequent move depending on the answer. Additionally, during runtime, ask-moves assigns variables and registers user interests. These updates influence subsequent dialog selection in both current and future sessions. Another move type is the **personalize** move, which generates a question based on the robot’s last question and the user’s reply, and a **Play** move that triggers non-verbal output, such as sounds or animations.

One mini-dialog consists of a sequence of dialog moves, optional dependencies, and metadata. Narrative dialogs are organized into narrative threads, each

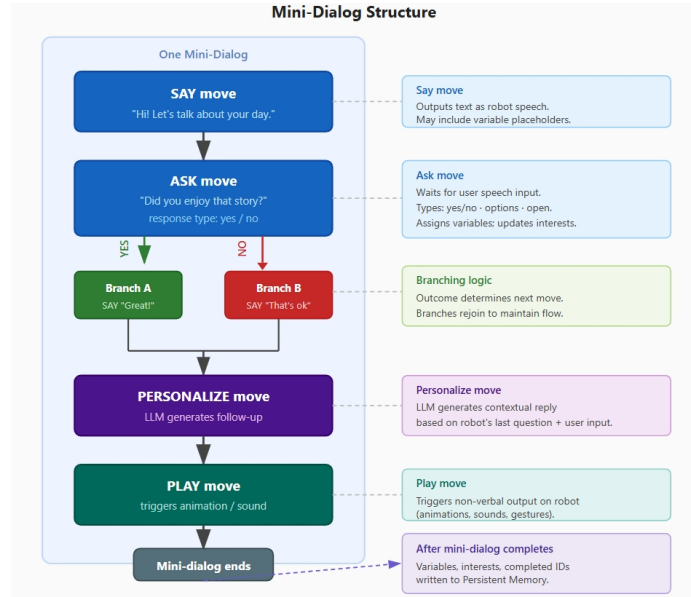


Fig. 2. Internal structure of a mini-dialog showing move types and branching logic

consisting of an ordered sequence of mini-dialogs that provides structure for the progression of a storyline. This structure supports continuity and familiarity over time. If the preferred narrative thread has no available progression, the next runnable thread is selected to maintain conversational flow. Chitchat dialogs are organized around topics and can be inserted between narrative elements. They are prioritized based on topic overlap with user interests or conversational context. Functional dialogs provide consistent structure at the beginning and end of a session. This way of organizing the mini dialogs enables the system to maintain narrative coherence while allowing for conversational variation and personalization.

4.3 Dialog execution pipeline

The system operates through a runtime execution pipeline that coordinates dialog execution, user input processing, memory updates, and dialog selection. Interaction begins with executing a move within a mini-dialog. Moves are processed sequentially. When an Ask move is executed, the system waits for a response. User input is captured through speech recognition and mapped to structured response types. These structured outcomes are used to determine the next step in the dialog. Each Ask move defines a set of possible outcomes. Based on the interpreted response, the system selects the corresponding branch and continues execution. This allows the dialog to adapt locally while preserving the authored structure. After each interaction, relevant information is stored in memory, in-

cluding user-provided variables, detected topics of interest, completed dialog units, and interaction events.

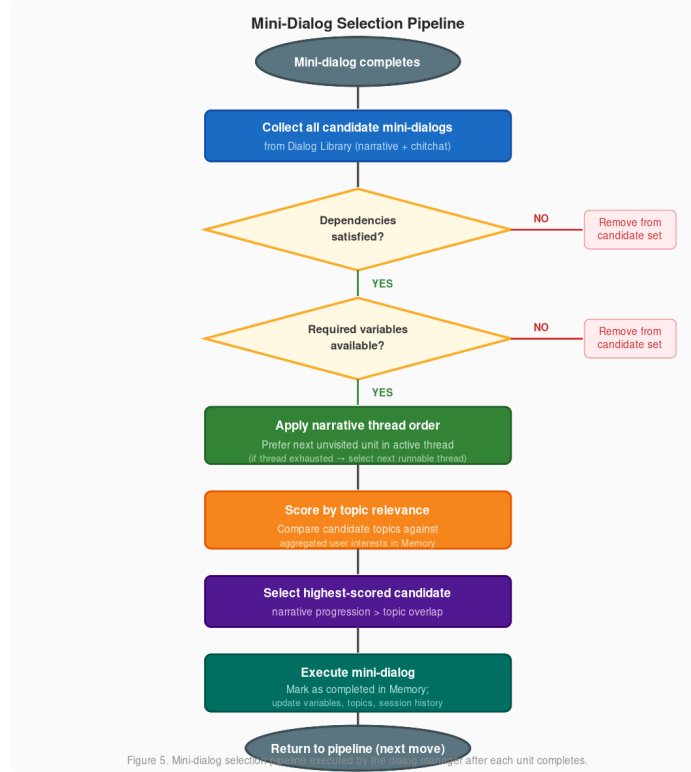


Fig. 3. Mini dialog selection pipeline

When a mini-dialog is completed, the dialog manager selects the next unit to execute. Selection is based on narrative progression, dialog dependencies, and topic relevance. The dialog flow is managed through rule-based scheduling and dependency validation. A dialog unit can only execute when its dependencies are satisfied and required variables are available.

Sessions are constructed by choosing a sequence of mini dialogs. In the current system, sessions follow the predefined agenda: greeting → narrative → chitchat → narrative → chitchat → farewell. However, the agenda is not fixed and can be tailored to the desired interaction flow and application context.

After running the farewell, the session ends, the history is saved in the participant file, and can be continued in a future session.

5 Methodology

To answer the research questions, we split the evaluation into 2 stages. First, an internal technical evaluation assesses the performance and robustness of the implementation. Second, experts familiar with similar systems assess the usability and extensibility of the multi-session narrative dialog system for future development. The technical evaluation precedes the expert evaluation so that usability findings reflect design limitations rather than incidental runtime failures.

5.1 Stage 1 - Internal technical evaluation

Evaluation objectives. This stage ensures that the narrative dialog system is technically stable before involving external participants. The aim is to establish a quantitative performance baseline and identify where the pipeline crashes.

System setup and Test environment Evaluation was conducted by one researcher in a controlled setting, using a set of predefined mini-dialogs. All service integrations were active during testing.

Test protocol We evaluate the system (see Section 4) to check if the implementation runs complete mini-dialog cycles (say → ask → response → next move) without manual intervention. We also check if the authoring tool can load or edit dialogs and execute them as intended. During runs, the system behavior was monitored and technical issues were logged.

Measurements Several types of data were collected during the evaluation. System-generated error logs were collected to identify runtime failures, execution errors, and debugging events during task execution. In addition, the system produces session history files for each participant containing the executed dialog moves, responses, and interaction events. These files provided insight into dialog execution and branching behavior.

Robustness and Error handling tests. We add noise and deliberate mispronunciations to the speech recognition pipeline to confirm that the system falls back to don't-know/fail branches and retries rather than crashing. This tests robustness to errors.

Functional validation: Tests are applied to all dialog components, checking for structural errors and correct variable substitution and branch resolution. Session history files are inspected to confirm that transitions and errors are accurately recorded.

Dialog coverage and State consistency: All reachable paths per mini-dialog (yes/no/silence; option choices; success/fail) are generated and checked for dead-ends. This guarantees that every authored branch is executable and prevents runtime stalls or unreachable content. Multi-session simulations verify that user-model variables, dialog identifiers, and topics of interest persist correctly across sessions, so that narrative dependencies are neither skipped nor repeatedly triggered.

Content and Output validation: Edge-case inputs are used to probe open-ended and LLM-generated responses for contextual appropriateness. Topic extraction accuracy is evaluated by looking at the extracted topics with the LLM function and reporting the precision/recall and uniqueness rate. This confirms

that selecting chitchats by topic is meaningful. Long-run tests measure system stability and resource usage under repeated execution.

Response time is also measured because delays affect user engagement and social expectations [1]. It ensures natural interaction and flags regressions when services slow down.

5.2 Stage 2 - Expert informed protocol

Evaluation objectives. In this study, expert interviews were used because they allow for detailed feedback on both the interaction experience and the system’s technical or authoring-related properties. They are expected to design, modify, or extend dialog structures and the overall system.

Participants Six experts participated in the evaluation. Five were PhD researchers in the field of HRI, all with prior experience working on similar dialog systems. Their technical background allowed them to evaluate both the authoring workflow and the system architecture in depth. The sixth participant was the writer expert who authored the mini-dialogs. He brought direct familiarity with the system from a content creation perspective. His involvement ensured that writers’ needs and desired authoring features were represented.

Interview procedure. The interview consisted of 5 phases: an initial interview, a small tutorial, including a demonstration of how the system works (where the participant interacts with the system), a task-based evaluation based on the interaction they just had, a codesign activity, and a post-questionnaire.

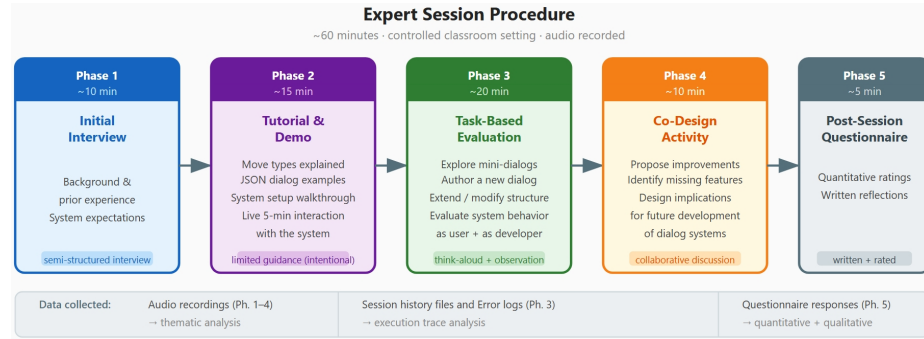


Fig. 4. Expert session procedure

The interviews were conducted in an empty classroom at the university. Each session lasted approximately one hour and followed the same structure and task sequence (see Figure 4). The full question list is provided in Appendix B.

Phase 1. Before interacting with the system, participants were asked a short set of questions to establish their background and expectations. These questions explored their experience with dialog systems and the contexts in which they might use such systems. Participants were also asked to describe what they

considered essential requirements for a narrative dialog system and how they expected the system to operate.

Phase 2. Then the system and the authoring workflow were explained in a short tutorial. The move types were described; extra focus was given to the LLM move explanation. Participants saw example dialog JSON files and were shown how to run the system. These explanations were intentionally limited to test whether new users could understand the system without extensive external guidance. A short five minute interaction with the system was also run. This way, the users understood what the system is capable of.

Phase 3. Next, participants completed a sequence of tasks designed to evaluate different aspects of the system. Participants first explored existing mini-dialogs to understand how conversations are structured and how narrative elements are combined within a session. Participants then created or edited a mini-dialog using the authoring tool. This task evaluated whether the dialog format, schema, and editing workflow were understandable and usable.

Participants were also asked to propose extensions to the system, such as a new LLM feature, branching option, or dialog move. Due to time constraints these were not implemented during the interviews. Instead, the task served to test the extensibility of the system and to identify what features developers need.

Phase 4. Following the task-based evaluation, participants took part in a short co-design activity to identify the system’s strengths, limitations, and potential improvements. The goal of this activity was to encourage participants to reflect on their interaction with the system. For each identified problem, participants were asked to propose possible improvements. Then, they had to prioritize one improvement they considered urgent to implement. This co-design activity provided structured qualitative insights into user priorities and design directions for future system development. They were also asked to prioritize improvements and reflect on which changes would have the greatest impact on the system.

Phase 5. Finally, participants completed a short post-session questionnaire designed to evaluate their overall experience with the system. The questionnaire consisted of Likert-scale questions (1 is strongly disagree, 5 is strongly agree) and open-ended questions covering several evaluation dimensions.

Data collection. Three types of data were collected. All sessions were audio-recorded to capture verbal feedback and task observations. Participants were informed of the recording and gave their consent before the session. Post-session questionnaire responses provided quantitative ratings. System-generated session history files, produced automatically for each participant, documented the executed dialog moves, user responses, branching behavior, and interaction events, offering an objective record of system behavior during tasks.

Analysis method. Qualitative data collected from interviews, questionnaire responses, and observations were analysed using inductive thematic analysis. Thematic analysis was chosen because it enables the identification of recurring patterns in qualitative feedback on usability, system design, and extensibility.

The analysis followed the general procedure described by Braun and Clarke [3] First, the data were reviewed and organized. The relevant data segments

were labeled with descriptive codes that represented recurring issues, design observations, or usability challenges. These codes were then grouped into broader themes representing recurring patterns in participant feedback. The themes were then reviewed and refined to ensure that each captured a pattern and that they were clearly distinct from one another. Finally, the themes were interpreted in relation to the research questions, and they are presented in the Results section.

6 Results

6.1 Internal evaluation results

The internal evaluation was conducted on 10 participant history files that produced 20 sessions and 122 mini-dialog executions in total. Each file had a range of 1-4 sessions. Each session has 4–7 dialog units following the predefined agenda.

Functional validation. All 20 sessions were completed without crashes or unrecoverable errors. Every dialog unit that was started reached a corresponding end event, confirming that the execution pipeline ran end-to-end as intended across all files and sessions, yielding a binary functional pass rate of 100% (20/20). Everything is logged in the session history files, and all interaction types (open, yesno, options, personification) were observed to complete and advance the dialog graph as authored.

Robustness and Error handling. Across 105 user responses, 5 were null (unrecognized or timed-out inputs), meaning a null rate of 4.8%. Failure rate varied by response type: options questions had the highest null rate (4/22, 18.2%), followed by yes/no questions (1/24, 4.2%), while open-response questions produced no failures (0/59). In all five cases the system triggered the fallback branch without crashing, confirming that error handling behaved as intended. No runtime crash or unhandled exception was recorded across any session.

Dialog coverage and State consistency. Of the 7 files that had multiple sessions, no dialog unit was repeated across sessions except one. Participant 1’s second session executed an identical dialog sequence because the session state had not been saved before the second run began.

Content and Output validation. The topic extraction function produced 53 topics across 10 participants (34 unique). The overall precision of extracted topics is $40/53 = 75.5\%$, and the mean per-participant precision is 77.2%. Qualitatively, the majority were semantically relevant (e.g., robots, waves, mountains, dogs, crabs, forest, birds, dragon). However, a subset consisted of function words or low-information terms (e.g., have, let, went, two, how), indicating that the LLM extraction does not consistently filter stop-words and filler language.

Session duration data suggest that the LLM personalization component adds approximately 80–110 seconds per session relative to equivalent sessions without it, as shown in figure 5. The shortest sessions correspond to dialog sets with fewer interaction turns, while longer sessions include dialog that ran an LLM personalization move. There were no sessions with a noticeable delay in service latency.

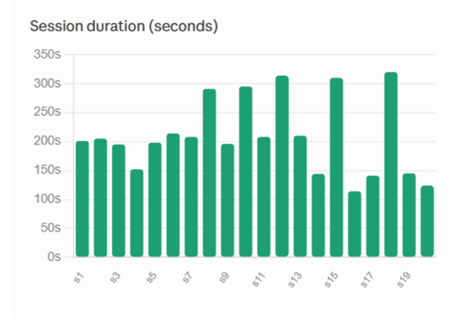


Fig. 5. Session duration in seconds

6.2 Expert interviews results

The thematic analysis revealed 8 interrelated themes, each with several sub-themes describing more specific aspects of the interaction and system design. Because participants engaged with the system in a dual role, the themes naturally cluster into two perspectives.

The developer-focused themes describe design and development needs, including the use of LLM moves, hybrid authoring workflows, and debugging needs. This includes system architecture, tooling, and implementation constraints that shape the interaction indirectly through design choices.

The user-focused themes describe interaction challenges and how users experience the conversation. They include dialog sequencing and transitions, recognition and turn-taking behavior, conversational agency and leadership, pragmatic intent mismatches, inflated expectations, and the role of robot identity and contextual framing.

6.3 Theme 1: LLM moves

During interviews, participants consistently described LLMs as an extension of the dialog system that can improve adaptability, safety, and personalization. The LLM move already implemented generates responses based on the user’s most recent input. Participants described it as the most straightforward LLM integration.

LLMs for dialog flow. Participants emphasized that LLMs could also be used to navigate the dialog structure itself. Instead of structured sequential execution, the system could use the user’s answer to move through the session in different ways, by playing with the dialog sequence and selecting better next mini-dialogs or branches (based on the content of the response), making it “more dynamic and different per interaction”. Participants proposed the use of LLMs to extract user interests from open-ended responses. Interest extraction was described as a foundation for later personalization (using interests to choose future chitchat).

LLMs receive full context. Several argued that using the full conversation history, including prior sessions, would enable more coherent personalization and

reduce repetition. The writer described how, in some experiments, entire scripts were embedded into the system prompt so that the LLM-generated segments remained coherent with what had already occurred.

Generate within narrative building blocks. The writer proposed a specific LLM move that operates within constrained contexts. He suggested structuring the conversation into meaningful narrative building blocks (introduction, context setting, clarification of goals, narrative exchange, and closure) and calling an LLM to generate within the specific block.

LLM checking on another LLM as safety mechanism. Another main point was that LLMs should have a safety mechanism. Expert 3 proposed a second-pass LLM to evaluate appropriateness and safety, especially in sensitive contexts involving children or sensitive data. It was described as an LLM checking on another’s LLM’s generation or a form of self-grading mechanism: “Generate a question based on what the child said, and then have the output go to a second LLM asking: is this content appropriate?; does it make sense given the context?; Are there any swear words or inappropriate language / does it pass the filter?” For each question, the LLM gives a grade, and if the grade is low, it will re-generate a response.” Some interviewees further emphasized privacy concerns of 3rd-party LLMs. To detect personally identifiable information (like name, address), we should already do a precheck with an in-house LLM, then send it to the self-grading mechanism. A possible problem is that the back-and-forth LLM checking introduces latency. The writer introduced a dramaturgical solution. He argued that response timing can be narratively justified: “If you create a story about an old robot... [it] is giving you reasons and empathy for a robot that is a little bit slower.”

Trigger words to switch to scripted segments. Another way to safeguard the system in case the LLM hallucinates is to implement a trigger word that, if said by the robot’s supervisor (during experiments), will force the LLM interaction to end and go back to pre-scripted moves. The writer described that in one experiment of a similar project, the actor could terminate the generative interaction and return to the script if it became unproductive by using a trigger word. This preserves narrative coherence when generative output fails.

6.4 Theme 2: Hybrid Authoring Tool

This theme focuses on how dialog systems are authored and maintained. For this thesis, writers produced dialog in natural, script-like formats in PDFs, which then had to be converted to the JSON structure required by the dialog system. Expert 5 emphasized that most writers prefer to work in natural language: “Creative people like to be creative in the way they normally express themselves.” In practice, all participants used LLMs as a conversion tool. One expert emphasized the need to develop a reliable prompt or a Python-based tool that could consistently perform these conversions correctly, with no extra effort from the person managing the JSON conversion.

Check for unreachable mini dialogs. To further help creators, experts emphasized that a hybrid authoring tool should provide visual or graph-like repre-

sentations to display branching paths and find unreachable nodes or disconnected dialog segments. Similarly, Expert 5 argued that creators need a clear overview of flow and breakpoints. He then proposed simulated testing using an LLM as “infinite user” with configurable characteristics (e.g., age, interests) to stress-test dialog structures: “then you have an automated way of evaluating your designs to see where it breaks.”

Lightweight annotations. Several participants suggested that the system should require minimal extra work from writers, provide small cues that make automatic conversion more reliable, and avoid forcing writers into technical schemas. Expert 4 proposed that writers could add simple markers, such as question types or explicitly listing the options the user has to choose from. The writer stated that he would like the ability to annotate tone and add paratext.

Need for a web interface. Moreover, a web interface was proposed to turn script uploads into a readable structure and allows line-level editing and on-the-spot configuration. This web interface would also help generate new, similar mini dialogs by analogy. For example, if a script handles a conversation about dogs, the tool could produce a structurally equivalent segment about giraffes. So, the LLM interaction is guided by the original author’s intent.

The writer noted that dialog lines were frequently adjusted during live testing sessions, indicating that authoring tools must remain flexible. This indicates that authoring support cannot be treated as a one-time script-to-JSON pipeline. Instead, tools must support rapid iteration and fine-grained edits during development and testing phases.

Scripted and LLM systems require different writing strategies. The writer made a clear distinction between writing for branching systems and for LLM-integrated systems. If the system relies on pre-scripted branching, he recommends “write as if for a theatrical play.” This implies anticipating possible human responses and scripting structured dialog in advance. In contrast, when integrating LLMs, writers should provide contextual framing, example dialogs, and goal-oriented building blocks that guide generative continuation.

Expert 5 argued that a fully LLM-based system would introduce several risks, including persona drift and overly question-driven conversational patterns. Even when using system role instructions, the robot’s identity may gradually shift toward a generic “friendly, helpful chatbot”.

Some writers have creative ownership concerns when using AI tools. Expert 4 highlighted that adoption of LLM-based tools may encounter resistance within creative communities. Based on discussions he had with writing students, he reported skepticism toward AI tools due to concerns about creative ownership, environmental impact, and training data usage. He noted that some writers explicitly avoid LLMs, preferring not to have their input used for model training. This suggests that hybrid authoring tools should allow creative practitioners to remain in control of their process and use AI support only if they feel comfortable doing so.

6.5 Theme 3: Debugging & Developer mode

Long mini-dialogs made repeated testing time-consuming, particularly when developers wanted to verify a single connection or transition within a larger flow. As one participant noted: “I would like to skip through things in a developer mode... not have to go through the whole interaction (approximately 20 minutes), to see if it works.” (Expert 3) Participants suggested practical recovery mechanisms, such as a keyboard shortcut to skip to the next line or fast-forward to a specific dialog node to make debugging more efficient.

Participants also reported situations in which the system became stuck mid-session, requiring a full restart. In some cases, restarting caused the entire mini-dialog to reset rather than continue from the previous state. This unpredictability made error recovery fragile and slowed down iteration.

6.6 Theme 4: “What comes next”

Participants consistently highlighted that the perceived naturalness of the interaction depends on how the system decides what comes next and how topic shifts are connected. Participants described the current selection approach as too random. They proposed strategies based on user interest signals, conversational history, and narrative structure.

Selection based on user interests and engagement. Expert 2 argued that user interest signals and engagement should drive selection. She suggested using response length as a practical signal (one-word answers as low interest; elaborate answers as high engagement) Expert 2 also mentioned potential lightweight NLP cues, for example: “to see if [the child] is sarcastic”.

Participants also emphasized that the system should take into account a broader conversational history. Expert 5 suggests that the system should exploit the conversation’s prior state and the user’s previous choices to estimate which mini dialog is most likely, thereby preserving narrative coherence. While acknowledging that occasional randomness can introduce variety, he emphasizes that meaningful continuity depends on heuristics derived from the interaction history rather than pure chance.

Expert 5 also proposes evaluating simpler heuristic approaches alongside full probabilistic models to evaluate different methods, using user feedback to compare outcomes. For adult participants, he recommends asking: During the interaction we have chosen this mini dialog for you. Looking at this list of other options, which one would you prefer and why? thereby gathering qualitative data on the adequacy of the selection method. He expects that, based on those answers, we can extract indicators or factors that could contribute to a better and meaningful choice rather than random. However, he emphasizes that soliciting explicit preferences from children is challenging because they often lack the ability to articulate their choices.

Nevertheless, writers and designers could be evaluators by presenting them with what the system chose as the next mini dialog and asking whether it aligns with their creative intent: “the system would have chosen this one, but from your experience would that have been your choice ... and why. “

Unmotivated transitions feel abrupt even when topics are relevant.

The experts reported that abrupt topic changes often made the conversation feel random, even when the selected topic was relevant. Several experts described confusing jumps, for example when a conversation about dreaming suddenly shifted to unrelated chitchat about nature before returning to the original topic. As Expert 1 explained, in human conversations, people may momentarily diverge from a topic because something related comes to mind, before returning to the original subject. She stated that chitchats should be connected to the previous topic. For example, if the user mentions “scary dreams,” the system could briefly discuss fear as a chitchat before returning to the main storyline about dreams. Participants accepted the idea of chitchat as “natural distraction,” but the choice needs to feel motivated or triggered by something in the conversation rather than introduced arbitrarily.

Building on this observation, Expert 1 emphasized that transitions in the current system often felt too abrupt and lacked justification. Simple phrases such as “let’s talk about something else” were perceived as insufficient because they did not explain why the topic change occurred. As she noted, “having maybe a good transition might even be more important than choosing which mini dialog should come next.” Expert 1 proposed that LLMs could help generate these short transition sentences that explain such shifts and make the conversation feel more coherent.

6.7 Theme 5: Recognition & Turn-taking problems

Users adapt their behavior to match the system’s rhythm. During the interaction, participants described actively adapting their own speaking behavior in response to the system’s timing, recognition constraints, and feedback cues. Therefore, recognition and turn-taking behavior directly shaped how natural the conversation felt. One expert described mirroring the system’s rhythm and shortening answers after noticing that longer responses were not handled well.

Fixed timeouts and cutting off users while resuming speaking damage the interaction. Participants linked being cut off mid-sentence to the system’s seven-second listening window, which moves on regardless of whether the user had finished speaking or wanted to continue. This was a major contributor to feeling unheard, especially for users who prefer to elaborate. The writer identified the following sequence as particularly harmful: the user pauses, the robot starts saying its next move, the user resumes his initial thoughts, but the robot continues anyway because it has already moved to the next step. As the writer emphasized, this experience “feels like the system is running a script that is pre-written” and is unresponsive to what users say. One solution is that the robot should remain interruptible. The expert explicitly suggested that if the user resumes speaking while the robot is talking, the robot should stop and listen again. Expert 4 specifically noted that children may pause to think and suggested that a short “hmm...” after a pause could grant more time.

6.8 Theme 6: Imbalance between system control and user agency

Participants consistently described the interaction as system-led. Users experienced the dialog as primarily structured around the robot asking questions and the user responding within constrained formats. Some noted that their expressed topic preferences were ignored and the system continued on its scripted path. As experts described, the interaction “felt like an interview”, where users cannot ask questions back or initiate topics. This asymmetry is a consequence of system capability. If the robot cannot respond to all user-initiated moves, it will naturally stay in the lead. Participants advocated for a more balanced dynamic in which either party can take the lead. The robot should be able to detect relevant triggers in user speech and temporarily follow the user’s direction while being grounded in its narrative role.

6.9 Theme 7: intent mismatch in natural conversation

Humor and elaborated answers are misread as invalid input. Multiple breakdowns occurred due to the system’s reliance on literal interpretations of user input. Participants frequently used humor, exaggeration, or narrative elaboration when responding, which the system failed to interpret appropriately. For example, playful or non-literal responses such as expressing interest in “bananas” or describing dreams involving “giant hotdogs” were not recognized as valid conversational contributions, causing the system to repeat questions or enter failed branches. Similarly, when participants attempted to shift topics (e.g., mentioning the weather), the system ignored these inputs and continued with the scripted narrative. These breakdowns were particularly evident in yes/no question formats. When participants provided elaborated answers rather than binary ones, the system often failed to recognize the intent of the reply, leading to repeated prompts, interruptions, or abrupt topic changes. Participants suggested explicitly labeling such questions, even though this reduces conversational naturalness: “So, yes or no” and then continue with the question.

Emotionally charged disclosures can trigger inappropriate responses. The writer highlighted the difficulty of handling unexpected or emotionally charged user responses. He described a case in which an elderly participant responded to “How are you today?” with “My husband died,” then the robot said, “Interesting, can you tell me more about it?” which is inappropriate. Although such situations were anticipated and pre-scripted in branching systems, the actual interaction still revealed limitations in pragmatic understanding. The writer expressed caution about delegating emotionally sensitive interactions entirely to LLMs. As a solution, he suggested that systems should rely on trigger-word detection: “ words like “dead” and similar emotional heavy words are detected in what the user says ... and specific responses are generated.” He noted that trigger-word strategies (e.g., detecting “died”) are imperfect because users may phrase it differently (“passed away”). This highlights a key issue. The system may process keywords correctly, yet fail to grasp the social and emotional meaning of an utterance. For the writer, adequacy in dialog is therefore not only about

semantic recognition but about context-sensitive, socially appropriate response generation, particularly in vulnerable settings such as care homes or hospitals.

6.10 Theme 8 : Robot identity and contextual framing

Robot identity must be established before dialog is authored. Narrative identity emerged as a structural design principle that shapes coherence, manage expectations, and tolerance of system limitations through story. Conversational success is therefore not determined solely by dialog management, LLM capability, or branching logic, but by how clearly the robot’s identity, purpose, and situational context are framed. The writer emphasized that before writing any dialog, the creative team first established a world in which the robot exists: Where is the robot? Who is the robot? Why is it here? What is its background story? What is the goal of the interaction? The robot’s presence must be narratively justified for the interaction to feel coherent. In earlier projects, as described by the writer, this included defining a robot identity aligned with the environment (e.g., hospital, care home, theatrical setting), sometimes accompanied by a “hidden agenda” such as teaching children something or offering distraction during hospital procedures. “the robot . . . help the children to forget a little bit that they are in a hospital.” According to the writer, this contextual grounding makes it logically and emotionally coherent that the robot is present and initiating conversation.

Identity can manage users expectation. The writer noted that users quickly attribute intelligence and emotional depth to robots: “When it talks to you, and it seems like it’s a little bit adequate, then immediately you think it’s much more intelligent than it is.” He warned that expectations can quickly exceed actual capabilities. As a result, “almost everybody . . . was immediately contributing too much to the platform,” leading to disappointment when the system fails to meet those projected abilities. Narrative identity can regulate user expectations. He highlights that the robot’s limitations can be tolerated if they are framed in terms of identity. Some reinforce the robot’s identity, making it more believable as a non-human agent. In addition, the expert described how even technical quirks (such as a robot consistently pronouncing numbers incorrectly by adding the decimals .0 every time) were eventually reframed as character traits rather than errors. “This robot is talking and saying numbers in that way . . . so why take it out? This is characterizing the robot” said the writer. In this case, what began as a flaw became part of the robot’s identity.

Robot’s input must be very good, output can be imperfect. The writer distinguished between forgivable imperfections and damaging ones: “There are some mistakes that are almost likable.” Imperfect or slightly “robotic” responses were considered acceptable, even characterful. In contrast, input failures, such as not understanding what the user says or interrupting them, are more damaging because they break flow and no amount of narrative framing repairs that.

Voice, gestures and embodied cues strongly shape perceived identity. Voice and audio characteristics contribute strongly to how the robot’s identity is perceived. The current system voice was described as too machine-like. The

experts recommended using external tools such as ElevenLabs to achieve more natural emotional inflections and better character alignment, particularly when the robot is meant to embody a specific persona. Voice, gestures, additional sounds, and animations are important for character coherence.

Explicitly framing limitations. The expert emphasized that robots can openly acknowledge what they cannot do, such as not being able to smell or taste. This transparency about limits was seen as beneficial: “If the robot says: I don’t know how that smells, can you describe it to me,” acceptance is preserved. Similarly, if a robot misidentifies a user’s gender, it can apologize and explain that this is difficult for it. Such acknowledgments reinforce the robot’s “robotness” while preserving trust.

7 Discussion

In this section, we will discuss the findings and prioritize future work based on what experts recognize as the biggest problems in having a quality interaction. We will address the third research question of how the system can be improved.

The 100% functional pass rate confirms system is stable enough for expert evaluation. Multi-session persistence functions correctly, validating that the memory architecture works.

Topic extraction has noise. Two failure modes were identified: extracting function words from answers, and ASR fragments passed through without filtering. We propose a part-of-speech filter on the extraction prompt and an ASR confidence threshold.

Sessions that had mini dialogs calling LLM personalization move are systematically longer, indicating that inference latency is the primary time cost and should be treated as a design parameter. This will affect the system when adding multiple LLM moves, as described in theme 1.

The system successfully provided a stable, controllable dialog structure that the participants understood. However, quality breakdowns emerged between mini-dialogs (transitions), between system and user turn (turn-taking), and between user intent and system interpretation (pragmatics). These point to a single systemic gap. The current architecture governs what the robot says and when, but does not yet model how the conversation feels from the user’s perspective. Participants adapted to this gap by simplifying their language and shortening responses, which is a well-known sign that a system is shaping user behavior rather than accommodating it.

A second key finding is that users interpreted all system behavior, including its failures, through the robot’s narrative identity. This has a double implication: a coherent identity increases tolerance for technical limitations, but it also raises the stakes when the system violates the expectations it has created. These two insights frame everything that follows.

7.1 Implications for system design

A summary of findings across themes is presented in appendix, Table 1. These findings have several combined implications for how the system should be improved, directly addressing RQ3.

First, the findings show that conversational coherence cannot be reduced to selecting a relevant next topic. Even when topic selection is reasonable, the interaction breaks down if the transition between topics is not explained, and breaks down further if the system does not accurately register what the user just said. This suggests that dialog sequencing, transition generation, and input understanding should be treated as a single design problem rather than as separate components: a transition that is well-justified but follows a recognition failure will not repair the user’s sense of being heard, and accurate recognition without a motivated transition will still feel arbitrary.

The dialog execution pipeline (Figure 1) illustrates how these issues concentrate at specific stages of the loop that the system repeats throughout a session, which is why improvements to sequencing, transitions, and input understanding need to be designed together rather than independently. Conversational quality emerges from the coordination of dialog structure, transition design, turn-taking behavior, pragmatic understanding, and narrative structure. Across themes, participants consistently favored a hybrid approach in which structured dialog provides coherence and control, while adaptive components, such as LLM-based moves, introduce flexibility within clear boundaries.

Second, the imbalance between system control and user agency (Theme 6) and the pragmatic intent mismatches (Theme 7) point to the same underlying limitation from two angles: the system can only be as responsive as its ability to interpret and act on what the user actually says. Giving users more conversational initiative is only meaningful if the system can recognize and respond to the topics, questions, or emotional disclosures that users introduce. This implies that improving user agency is not primarily a matter of redesigning dialog flow, but of extending the system’s capacity to interpret a wider range of user contributions, for example through LLM-supported intent interpretation operating within the existing “safe zone” model from Theme 1. Third, robot identity and contextual framing (Theme 8) function as a layer that interacts with all other findings rather than as an isolated topic. A coherent narrative identity does not remove the problems identified in Themes 4–7, but it changes how users interpret and tolerate them: the same delay, repetition, or imperfection can be read as a system failure or as a believable character trait depending on whether it is narratively framed. This means that identity and framing decisions should be made early and explicitly, since they set the interpretive context within which all other improvements will be perceived. Finally, the developer-focused themes (1–3) are best understood as enabling conditions for the user-focused improvements above, rather than as separate priorities. A hybrid authoring tool and a developer/debugging mode do not directly change what the user experiences, but they determine how feasible it is to design, test, and iterate on the sequencing, transition, recognition, and framing improvements that do.

7.2 Recommendations for future development

To answer the third research question, we used Figure 6 and 7 to prioritize the system’s main improvements. Figure 6 summarizes participant assessments across the themes and reveals which areas were most consistently rated as prob-

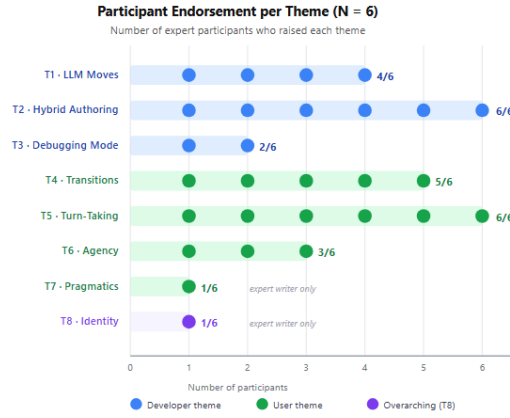


Fig. 6. Participant endorsement per theme

lematic. Figure 7 maps the resulting recommendations by impact and implementation effort. The recommendations are ordered from most to least important, reflecting both how severely participants experienced the issue and where it sits in the interaction pipeline. Problems closer to the core loop (recognition, intent, transitions, agency) were reported as more damaging to perceived conversational quality than issues confined to the framing layer (system persona) or to developer tooling.

R1 : Improve turn-taking. Fixed listening windows and the inability to re-enter a turn after an interruption produced an interview-like dynamic that participants found unnatural. This is an architectural issue because the current pipeline treats turn boundaries as system-controlled and requires changes to the turn management layer that are independent of dialog content. To solve this, the system should replace fixed listening windows with dynamic turn-taking: detecting pauses, allowing users to resume speaking, and re-yielding the conversational floor if the robot has already started its next move when the user resumes speaking. Participants were explicit that accurate input understanding matters more than polished output, making this the highest-priority change.

R2 : Resolve pragmatic intent mismatches. The system currently maps user input to predefined response types, which forces users to accommodate the system rather than the reverse. Humor, indirect language, and emotionally sensitive disclosures all fall outside this mapping and are handled poorly as a result. The fix is to layer LLM-supported interpretation on top of the existing structured dialog, consistent with the "LLMs as controlled components" framing from Theme 1, combined with fallback strategies that acknowledge pragmatically complex input without forcing a branch decision. This does not require replacing the structured pipeline; it extends it to handle input the current intent classifier cannot resolve cleanly.

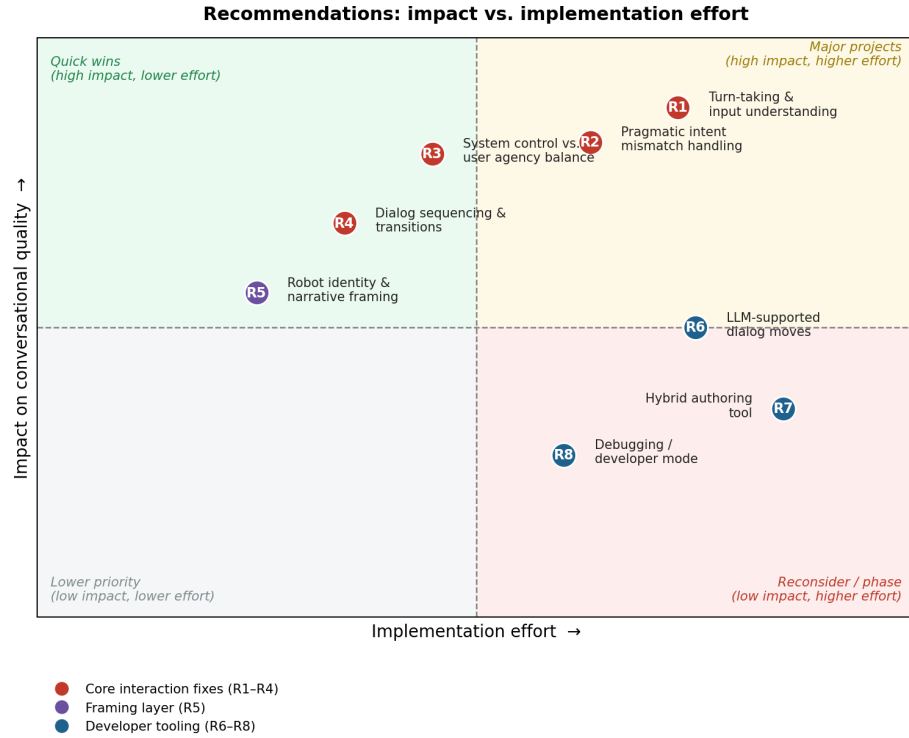


Fig. 7. The eight recommendations plotted by their impact on conversational quality and their implementation effort.

R3 : Rebalance system control and user agency. The system should allow users to introduce topics, ask questions, and have these acknowledged. In practice, this means the dialog manager needs to handle user-initiated moves and at minimum by recognising them and holding them until an appropriate moment, rather than defaulting to the scripted branch. This recommendation depends on R1 and R2: agency can only increase to the extent that the system can detect when a user talks and respond to what users introduce.

R4 : Improve dialog sequencing and transitions. Selection of the next mini-dialog should move from random selection toward a context-sensitive approach that considers user interest signals, engagement, and conversational history. The system must generate transition utterances that make the reason for a shift explicit, not generic phrases like "let's talk about something else." This can be done with a targeted LLM call given the previous mini-dialog, the next topic, and the user's last response, without exposing the broader dialog structure to open-ended generation. Structuring sessions into larger narrative blocks can further reduce fragmentation.

R5 : Strengthen robot identity and narrative framing. The robot’s identity, purpose, and situational context should be defined explicitly and early in the design process, since this framing shapes how users interpret the improvements above. Voice, tone, gestures, and other performance cues should be aligned with this identity, and known limitations should be reframed as character traits or openly acknowledged where appropriate, to manage user expectations.

R6 : Expand LLM-supported dialog moves. LLM-based moves should be extended for transition generation, personalization, and interest extraction, but must operate within predefined “safe zones” that have fallback mechanisms to keep generative behavior aligned with narrative goals. Participants raised concerns when the generative behavior was not constrained, as LLMs can hallucinate. A solution is to always give the LLM, the current narrative context, user history, the intended communicative goal. All output should pass a validation step, a lightweight judge model, before delivery, with authored fallback responses as a hard boundary. This recommendation underpins R2 and R4.

R7 : Develop a hybrid authoring tool. An authoring environment should be developed that lets writers work close to natural script form while supporting lightweight annotation, visualization of branching structure, and LLM-based “infinite user” testing to detect unreachable paths and breakdowns. It should also provide LLM-assisted generation of alternative dialog variants from authored examples, reducing the burden of manually scripting all branches while keeping authored content as the authoritative source. This tool is what would make R4–R6 practical to design and maintain at scale.

R8 : Add a developer mode. Developers should be able to skip to specific dialog nodes, fast-forward through long mini-dialogs, and recover from stuck sessions without a full restart.

7.3 Limitations

This study has several limitations that should be considered when interpreting the findings. The interaction sessions were relatively short and conducted in a controlled environment, which may not reflect real-world conditions where noise, interruptions, and contextual variability influence interaction, affecting speech recognition, and turn-taking. Multi-session narrative dialog systems are designed to unfold over longer periods of interaction, where memory, personalization evolve. As the interaction with the system was limited to 5 min per expert due to time constraints, multi-session memory and narrative continuity, were not directly evaluated.

Also, the internal technical evaluation was conducted by a single researcher, limiting detection of latent implementation errors that might surface under different usage conditions.

7.4 Future work

Future work should first implement and test the prioritized improvements (see Section 7.2). An important next step is to evaluate the system with users in a longitudinal and less controlled setting. For example, children interact with social robots from a different perspective and may reveal other needs, engagement

patterns, and breakdowns that cannot be captured through expert feedback alone. Long-term deployment would also provide insight into how well the system sustains engagement and narrative continuity over repeated interactions. Other future research should focus on the effect of different narrative identities and the influence they have on tolerance of system limitations and user expectations.

8 Conclusion

Sustaining long-term interaction in social robots requires balancing coherence, adaptability, and user engagement. We presented a multi-session narrative dialog system that combines structured mini-dialogs with memory and LLM-based moves.

A key contribution of this work is the identification of design implications for narrative dialog systems. The findings show that conversational quality depends not only on dialog content, but on how mini-dialogs are sequenced, how transitions are realized, how accurately user input is understood, how conversational control is shared between robot and user, and how the robot’s narrative identity frames all of the above. The findings further highlight the role of LLM features, and hybrid authoring tools, in making these improvements feasible to design, test, and maintain. Together, this thesis provides clear directions for future development of similar narrative dialog systems.

9 Appendix

A Acknowledgment

Many thanks to the experts who took part in the interviews! Their time and insights were invaluable to this thesis.

B Interview questions

This appendix lists the questions used in the semi-structured expert interviews. Open questions were answered verbally; closed items were rated on a five-point Likert scale (1 = strongly disagree, 5 = strongly agree).

Part A : Background and Context

1. What is your experience with this system?
2. In what context do you need the system? How would it be useful to you?
3. What are the main requirements that the system should meet? (*Open text.*)
4. What do you think the general functionality of the system is?
5. **Preference for representation.** For authoring dialogues, what do you generally prefer?
 - (a) GUI / visual graph editor
 - (b) Text-based formats (JSON, code)
 - (c) A mix of both
 - (d) No strong preference
6. Which parts are fully scripted vs. open/improvised? Why? How do you decide? Do writers say?
7. Walk me through your ideal workflow from “idea” to “robot dialogue”.

8. What would you want an authoring tool to do automatically (e.g., chunking into mini-dialogs, detecting questions/options, adding transitions)?
9. (*Open.*) Without running the system yet, describe how you think the system should select the next mini-dialogue. Based on what?

Part B: Co-Design Activity (After Tutorial)

1. If you could change one thing in the authoring workflow to make it smoother, what would you change?
2. How would you make the transition feel motivated rather than random?
3. If you were asked to implement a small extension to the system, such as an LLM-based extension, what would you implement?
4. What could be another LLM feature you would create?
5. What would you never delegate to an LLM?
6. What are the system's strengths, problems, and possible improvements?

Part C : Overall System Experience

1. I found the narrative dialog system easy to use overall.
2. The system met my expectations for a narrative dialog framework.
3. How was the interaction with the agent? Did it feel like an interview or was it a balanced dialog? Did you have enough space to talk? (*Open.*)
4. The JSON schema for mini-dialogs is clear and easy to edit.
5. I understand the difference between narrative, chitchat, and functional mini-dialogues.
6. Do you think a trained AI would be faster and better at authoring the file if we give the English dialog transcript?
7. I think I would like to use this system frequently for narrative dialog design.
8. I found the system unnecessarily complex.
9. After this session, I could independently design a new multi-session narrative.
10. I would need technical support to be able to use this system.
11. I would imagine most developers would learn to use this system quickly.
12. I needed to learn a lot of things before I could get going with this system.
13. I often had to mentally keep track of many details at once (IDs, branches, conditions, variables).
14. Overall, I felt satisfied with my interaction with the system.

C Key findings per theme

Nr. Theme	Key Finding
1 LLM Moves	LLMs are most valuable as constrained extensions to structured dialog systems for generating transitions, personalization within predefined “safe zones”, performing safety checks and extracting user interest.
2 Hybrid Authoring Tools	Writers need workflows that preserve natural script writing while supporting lightweight annotation, visualization of branching paths, and LLM-based “infinite users” for automated testing.
3 Debugging & Developer Mode	Long mini-dialogs make repeated testing slow; developers need to skip/fast-forward and recover from stuck sessions without a full restart.
4 What Comes Next & Transitions	Random topic selection and abrupt, unmotivated transitions reduce believability, even when the new topic is relevant.
5 Recognition & Turn-Taking	Fixed listening windows and failure to re-yield the floor after a user resumed speaking made participants feel unheard and led them to shorten and simplify their responses; accurate input understanding matters more than polished output.
6 Agency & Leadership	The interaction feels “interviewlike” because the system cannot respond to all user-initiated topics or questions so it has to stay in the lead.
7 Intent Mismatch & Pragmatic Understanding	Humor, indirect language, and emotionally sensitive disclosures are often interpreted literally, causing repeated prompts or inadequate responses.
8 Identity & Contextual Framing	A coherent narrative identity increases tolerance of limitations, manages expectations, justifies the robot’s behavior, and overall conversational acceptance.

Table 1. Overview of themes and key findings.

References

1. Jana Appel, Astrid von der Pütten, Nicole C Krämer, and Jonathan Gratch. Does humanity matter? analyzing the importance of social cues and perceived agency of a computer system for the emergence of social reactions during human-computer interaction. *Advances in Human-Computer Interaction*, 2012(1):324694, 2012.
2. Timothy Bickmore and Justine Cassell. Small talk and conversational storytelling in embodied conversational interface agents. In *AAAI fall symposium on narrative intelligence*, pages 87–92. Orlando FL, USA, 1999.
3. Victoria Clarke and Virginia Braun. Thematic analysis. *The journal of positive psychology*, 12(3):297–298, 2017.
4. Sandra Costa, Alberto Brunete, Byung-Chull Bae, and Nikolaos Mavridis. Emotional storytelling using virtual and robotic agents. *International Journal of Humanoid Robotics*, 15(03):1850006, 2018.
5. Rachel Gockley, Allison Bruce, Jodi Forlizzi, Marek Michalowski, Anne Mundell, Stephanie Rosenthal, Brennan Sellner, Reid Simmons, Kevin Snipes, Alan C Schultz, et al. Designing robots for long-term social interaction. In *2005 IEEE/RSJ*

- International Conference on Intelligent Robots and Systems*, pages 1338–1343. IEEE, 2005.
6. Lucrezia Grassi, Carmine Tommaso Recchiuto, and Antonio Sgorbissa. Knowledge-grounded dialogue flow management for social robots and conversational agents. *International Journal of Social Robotics*, 14(5):1273–1293, 2022.
 7. Daniel Green, Charlie Hargood, and Fred Charles. Use of tools: Ux principles for interactive narrative authoring tools. *Journal on Computing and Cultural Heritage (JOCCH)*, 14(3):1–25, 2021.
 8. Shengyue Guan, Haoyi Xiong, Jindong Wang, Jiang Bian, Bin Zhu, and Jian-guang Lou. Evaluating llm-based agents for multi-turn conversations: A survey, march 2025. *arXiv preprint arXiv:2503.22458*.
 9. Bahar Irfan, Sanna Kuoppamäki, Aida Hosseini, and Gabriel Skantze. Between reality and delusion: challenges of applying large language models to companion robots for open-domain dialogues with older adults. *Autonomous Robots*, 49(1):9, 2025.
 10. Bahar Irfan, Aditi Ramachandran, Samuel Spaulding, Dylan F Glas, Iolanda Leite, and Kheng Lee Koay. Personalization in long-term human-robot interaction. In *2019 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 685–686. IEEE, 2019.
 11. Sara Kiesler. Fostering common ground in human-robot interaction. In *ROMAN 2005. IEEE International Workshop on Robot and Human Interactive Communication, 2005.*, pages 729–734. IEEE, 2005.
 12. Liliana Laranjo, Adam G Dunn, Huong Ly Tong, Ahmet Baki Kocaballi, Jessica Chen, Rabia Bashir, Didi Surian, Blanca Gallego, Farah Magrabi, Annie YS Lau, et al. Conversational agents in healthcare: a systematic review. *Journal of the American Medical Informatics Association*, 25(9):1248–1258, 2018.
 13. Min Kyung Lee, Jodi Forlizzi, Sara Kiesler, Paul Rybski, John Antanitis, and Sarun Savetsila. Personalization in hri: A longitudinal field experiment. In *Proceedings of the seventh annual ACM/IEEE international conference on Human-Robot Interaction*, pages 319–326, 2012.
 14. Iolanda Leite and Jill Fain Lehman. The robot who knew too much: Toward understanding the privacy/personalization trade-off in child-robot conversation. In *Proceedings of the The 15th International Conference on Interaction Design and Children*, pages 379–387, 2016.
 15. Iolanda Leite, Carlos Martinho, Andre Pereira, and Ana Paiva. As time goes by: Long-term evaluation of social presence in robotic companions. In *RO-MAN 2009-the 18th IEEE international symposium on robot and human interactive communication*, pages 669–674. IEEE, 2009.
 16. Iolanda Leite, André Pereira, and Jill Fain Lehman. Persistent memory in repeated child-robot conversations. In *Proceedings of the 2017 conference on interaction design and children*, pages 238–247, 2017.
 17. Mike EU Ligthart, Mark A Neerincx, and Koen V Hindriks. Memory-based personalization for fostering a long-term child-robot relationship. In *2022 17th ACM/IEEE international conference on human-robot interaction (HRI)*, pages 80–89. IEEE, 2022.
 18. Maria Lombardi, Carmela Calabrese, Davide Ghiglino, Caterina Foglino, Davide De Tommaso, Giulia Da Lisca, Lorenzo Natale, and Agnieszka Wykowska. Would you let a humanoid play storytelling with your child? a usability study on llm-powered narrative humanoid-robot interaction. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 20066–20073. IEEE, 2025.

19. Dongxu Lu, Johan Jeuring, and Albert Gatt. Evaluating llm-generated versus human-authored responses in role-play dialogues. In *Proceedings of the 18th International Natural Language Generation Conference*, pages 20–40, 2025.
20. Mauliana Mauliana, Ashita Ashok, Daniela Czernochowski, and Karsten Berns. Exploring llm-powered multi-session human-robot interactions with university students. *Frontiers in Robotics and AI*, 12:1585589, 2025.
21. Nadun Ranasinghe, Wael M Mohammed, Kostas Stefanidis, and Jose L Martinez Lastra. Large language models in human-robot collaboration with cognitive validation against context-induced hallucinations. *IEEE Access*, 2025.
22. Merle M Reimann, Florian A Kunneman, Catharine Oertel, and Koen V Hindriks. A survey on dialogue management in human-robot interaction. *ACM Transactions on Human-Robot Interaction*, 13(2):1–22, 2024.
23. Ming Wang, Peidong Wang, Lin Wu, Xiaocui Yang, Daling Wang, Shi Feng, Yuxin Chen, Bixuan Wang, and Yifei Zhang. Annaagent: Dynamic evolution agent system with multi-session memory for realistic seeker simulation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23221–23235, 2025.